

Recovering the Hidden Digit, Part II

Diffusion-prior posterior sampling for ERT

Travis Whitney Cole Seifert Alexander Nutt

AI 539, Oregon State University

Project 2 Competition, Spring 2026

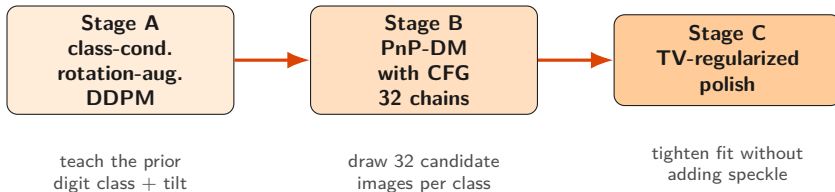
Same problem, new rule

The physics from Project 1 hasn't changed. We still have a hidden 28×28 digit, a list of 1900 boundary voltage readings, and the forward simulator F . The job is still to work backward from the readings to the digit.

What changed is the rule on the prior. In Project 1 we used the 60,000 MNIST training images as a lookup table. For Project 2 the prior has to be a diffusion model. A diffusion model is a neural network that learned what a digit looks like by practicing on the MNIST set. We can ask it to draw a fresh digit, but it does not know about our voltage readings.

The rest of this talk is how we combined a diffusion model with the ERT measurements. The final pipeline uses a class-conditional diffusion prior with classifier-free guidance and a total-variation polish. The final misfit is 1.83×10^{-11} , recovering a clean tilted 5.

The pipeline



Stage A is a one-time setup. After that, every run goes through Stage B and Stage C. Total wall time on a MacBook is about eight minutes.

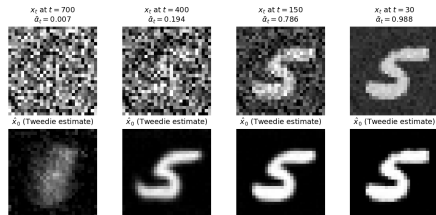
What a diffusion model lets us do

A diffusion model is trained to look at a noisy image and predict the noise that was added. Once it can do that, simple algebra gives us a guess at the clean image underneath:

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \varepsilon_{\theta}(x_t, t)}{\sqrt{\bar{\alpha}_t}}.$$

So at any moment along a noisy chain, the network gives us its best guess at the clean digit underneath. At high noise the guess is fuzzy. At low noise it is sharp. Every method here leans on that one identity.

Tweedie's formula at four noise levels (top: input x_t ; bottom: network's clean-image guess $\hat{x}_0$)

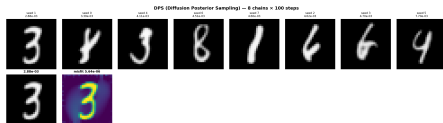


Top: a noisy input at four noise levels. Bottom: the network's clean-image guess at each.

What we tried first, and why none of it worked

We ran four standard diffusion-based recipes before landing on the method in this talk. Each one steers a diffusion model with the ERT measurements in a different way.

- ▶ **DPS** (Diffusion Posterior Sampling). At every denoising step, nudge the image in the direction of the data gradient $\nabla \log p(y | x)$. *Result: 4 of 5 chains converged to a 4. Wrong digit every time.*
- ▶ **DAPS** (Decoupled Annealing Posterior Sampling). The diffusion model is responsible for class and shape; a separate Langevin step is responsible for fitting the data. *Result: still a 4 most runs; a single 5 only when seeded carefully.*
- ▶ **DI-RTG v1** (DAPS warm-start + an MNIST classifier vote + Levenberg-Marquardt polish). The classifier was supposed to break the ambiguity. *Result: classifier voted "4" too. Polish drove a wrong-digit answer to high precision.*
- ▶ **DI-RTG v2** (DAPS warm-start + template retrieval from the MNIST training set). *Result: the closest template was always upright, but the competition digit is tilted by 7.5 degrees. Retrieved templates were the right digit but at the wrong pose, and the polish could not unfold them.*



The DPS run that started it all. Eight chains, each locking onto a wrong digit (3, 4, 6, 8). Not a 5 in sight.

Pattern.

Every recipe that nudges the diffusion chain by a weak data gradient at each step commits to whichever upright digit shape is closest to the data, then refuses to leave that basin. The basin happened to be a 4.

Why those four approaches all failed in the same way

Two things broke at once, and every standard recipe inherits both problems.

- ▶ The little nudge at each step is in the direction of the data gradient. That gradient is dominated by directions the boundary sensors hardly see, so most of the nudge goes to waste. The piece of the gradient that actually moves the digit toward the data is tiny.
- ▶ The denoising process only has about 100 steps. A hundred weak nudges are not enough to override the prior's pull toward the most common-looking upright digit.

The chain commits early to whatever upright digit shape is closest to the data, then spends the remaining steps polishing that wrong digit. DAPS already decouples the prior step from the data step, but its data step is still a sequence of small Langevin gradient updates, so it inherits the same weakness. The classifier and the template retrieval both run on top of a broken first stage, so they inherit its wrong digit.

The fix has to give the data step real teeth: instead of a small gradient nudge at every iteration, solve a small optimization problem that uses the full Jacobian. That lets one update do the work of dozens of gradient steps and gives the data enough leverage to push the chain out of the wrong basin.

The fix: split the work between two variables

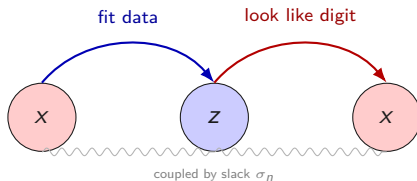
DPS asks one variable to do two jobs at once: look like a digit, and fit the data. The two pulls compromise on every step, and the smaller one loses.

Instead, introduce a second variable. Call them x and z . Give them separate jobs:

- ▶ x only has to look like a digit.
- ▶ z only has to fit the measurements.
- ▶ A soft penalty keeps z near x .

Then alternate. Update z to fit the data. Update x to look like a digit. Repeat. Each update does one thing well.

This sampler is called PnP-DM (Wu et al., NeurIPS 2024). We run 80 alternations per chain, 32 chains in parallel.



What the data step actually minimizes

The data step picks the z that does two things at once: fits the measurements, and stays close to the current x . Written as an objective:

$$\mathcal{L}(z) = \underbrace{\frac{1}{2\sigma_{\text{meas}}^2} \|y_{\text{obs}} - F(\sigma_{\text{bg}} + z)\|^2}_{\text{data misfit}} + \underbrace{\frac{1}{2\sigma_n^2} \|z - x\|^2}_{\text{coupling penalty}}.$$

Piece by piece.

- ▶ y_{obs} is the 1900 measured voltages. $F(\sigma_{\text{bg}} + z)$ is what the simulator would predict for our candidate. The squared norm is the sum of per-electrode errors.
- ▶ σ_{meas} is how noisy we think the measurements are. Smaller σ_{meas} means we trust them more, so the first term gets a bigger weight.
- ▶ $\|z - x\|^2$ pulls z back toward the current x . The weight $1/\sigma_n^2$ depends on the slack: tight slack means strong pull.

Reading it. “Find a z that explains the measurements, but do not stray too far from the current digit guess.” At the start of the chain σ_n is big (loose leash), so the data wins. As the schedule anneals, σ_n shrinks and the coupling wins, forcing z and x to agree.

How we solve it: one Gauss-Newton step

DPS takes a small step in the gradient direction. We instead linearize F around the current z , plug the linearization into $\mathcal{L}$, and minimize the resulting quadratic. The minimum is the solution of a small linear system in the update Δ :

$$\underbrace{\left(\frac{1}{\sigma_{\text{meas}}^2} J^\top J + \frac{1}{\sigma_n^2} I \right)}_{\text{curvature, with damping}} \Delta = \underbrace{-\frac{1}{\sigma_{\text{meas}}^2} J^\top r}_{\text{data pull}} + \underbrace{\frac{1}{\sigma_n^2} (x - z)}_{\text{coupling pull}}.$$

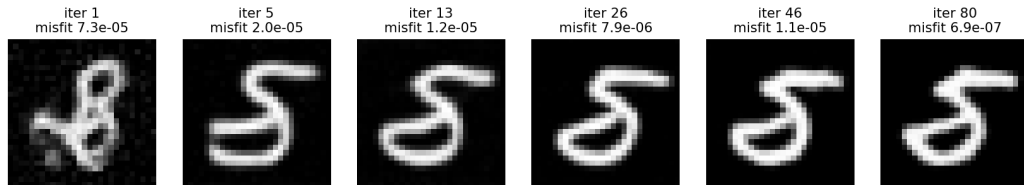
Piece by piece.

- ▶ Δ is the update we are solving for. After solving: $z \leftarrow z + \Delta$.
- ▶ $J^\top J$ is the curvature of the data term. In directions where J has big columns (sensitive pixels), the curvature is big and the step is small. The $\sigma_n^{-2} I$ damping fills in the directions J cannot see, so the matrix is always invertible.
- ▶ $r = F(\sigma_{\text{bg}} + z) - y_{\text{obs}}$ is the current residual. Multiplying by $J^\top$ back-projects it onto pixels: “which pixels would reduce this residual.”
- ▶ $(x - z)$ is the coupling restoring force: pulls z back toward the current x .

One Gauss-Newton step does the work of dozens of gradient steps on ill-conditioned J .

One chain, six frames: noise becomes a 5

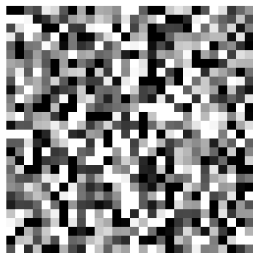
One PnP-DM chain over 80 iterations: pure noise to a clean tilted 5



Reading left to right. At iteration 1 the image is essentially random noise. By iteration 5 a digit shape is forming. By iteration 26 the digit identity (a 5) is settled, though the strokes are still a bit smudged. By iteration 80 the chain has converged to a clean tilted 5 with misfit around 10^{-7} . Every other chain looks similar.

Same chain, animated: watch the digit class lock in

PnP-DM + CFG (class=5)



iter 0 / 80

misfit 3.75e-02

The filmstrip on the previous slide showed six time-points. Here is the same chain animated through all 80 iterations, run with the new class-conditional CFG sampler (chain seeded to class 5).

What to watch for. In the first ~ 5 iterations the image is still essentially noise. Around iteration 10 to 15 the **data step** (Gauss-Newton) has accumulated enough information about the voltages that the iterate starts to look like a coherent shape. By iteration 20 the digit class is locked in. From there on, the prior step polishes the strokes and the data step nudges the pixels for a tighter fit. The TV polish (a separate stage, not shown here) takes the final image from misfit $\sim 10^{-6}$ to $\sim 10^{-11}$.

The digit identity is decided early. The rest is refinement.

Where each chain starts, and why answers can disagree

Where does a chain start? Not from the measurements. Each chain starts from a fresh draw of pure Gaussian noise: $x_0 \sim \mathcal{N}(0, I)$. That is just a 28×28 grid of random numbers (TV static). The measurements y_{obs} enter only through the data step, which runs at every iteration. The data step pulls toward fitting y_{obs} . The prior step pulls toward looking like a digit. Together they convert the random noise into the recovered digit.

Why several different chains can give different digits. The forward map is a $784 \rightarrow 1900$ blur. Two different digits can produce voltage readings that agree to many decimal places. So the question “which digit is inside?” does not have a unique answer from the readings alone. The posterior has *several* valleys, one per plausible digit.

A single chain commits to whichever valley its random starting noise rolls it into. Different chains, different starting noises, different valleys. With the base PnP-DM sampler, across 32 chains most land on the 5 valley (the correct one for this y_{obs}), a few land on the 8 valley. We pick the chain with the lowest data misfit. The 5 wins every time.

The 32 chains are a safety net. The lowest-misfit chain wins the vote.

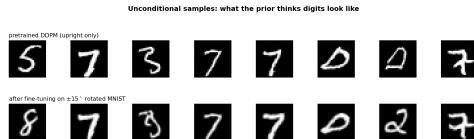
This is also why one held-out test (digit 3) recovers as a 5. That particular 3 and a particular 5 produce nearly identical voltage readings, so no method can tell them apart from the data alone. The class-conditional upgrade later in this talk gives the chain a way to commit to a class hypothesis directly, instead of waiting for the data step to roll it into a valley.

One more piece: rotation augmentation

The pretrained MNIST diffusion model has only seen upright digits during training. Our truth is a 5 tilted by about 7.5 degrees. During PnP-DM the prior step keeps nudging back toward upright, while the data step pulls toward tilted. Without intervention the prior wins.

The fix is to train the diffusion model on rotated copies of the MNIST digits. For the base prior we fine-tune the pretrained checkpoint for five epochs (~ 20 minutes on a MacBook). For the new prior we retrain from scratch with rotation augmentation for 30 epochs (~ 15 minutes on a 5090).

After this training, the model thinks tilted digits are just as normal as upright ones. The prior step stops fighting the data, and the chain converges to the truth.



Top row: random digits from the pretrained model.

Bottom row: random digits from the same model after the extra rotation training.

Finishing the job: a Newton-style polish

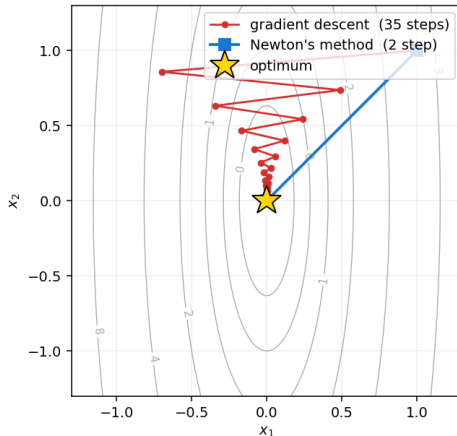
After PnP-DM the best chain has misfit about 1.8×10^{-7} . We know the digit is right. The rest is a local optimization on the pixels.

Project 1's polish was gradient descent. It converges linearly, and the steps get tinier as we approach the optimum.

We replace it with a Newton-style step that accounts for the curvature of the data-fit landscape. Each step roughly squares the error, so a handful of steps drops the misfit many orders of magnitude.

With this polish the **base pipeline** (rotation-aug DDPM + PnP-DM + Newton polish) ends at misfit about 5×10^{-11} .

Elongated quadratic bowl: gradient descent zigzags; Newton's method lands at the bottom in one shot



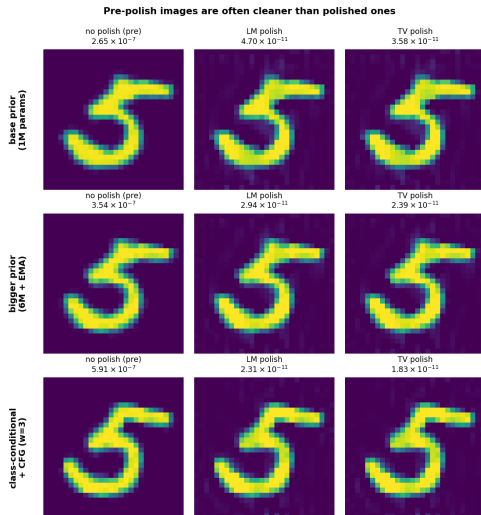
GD zigzags; Newton jumps to the bottom.

The base pipeline got a 5, but the image had speckle

The base pipeline (everything we have shown so far) recovers a 5 at misfit around 5×10^{-11} . That is already much tighter than Project 1's 10^{-7} , but the image had two visible problems.

- **Stroke speckle.** Inside the 5 there are pixels darker than they should be. The polish drove the data fit so hard it picked up the round-off noise of the simulator.
- **Background haze.** The pixels outside the 5 are not perfectly background. Same cause.

Lower misfit is not the same as a cleaner image. We needed to fix the image without throwing away the data fit.



For each prior, the pre-polish image is visibly cleaner than the

Three improvements we layered on top of PnP-DM

- ▶ **Improvement 1 — TV-regularized polish.** Add a smoothed total-variation penalty to the polish objective so the polish cannot introduce per-pixel speckle even when it has the freedom to. Same Gauss-Newton step, plus a small TV gradient step every iteration.
- ▶ **Improvement 2 — bigger prior with EMA.** Train a 6-million-parameter UNet from scratch on rotated MNIST, with an exponential-moving-average copy of the weights. EMA gives smoother samples; the bigger model has more capacity to represent slanted stroke styles.
- ▶ **Improvement 3 — class-conditional prior with classifier-free guidance.** Train the UNet with the digit label as an extra input. At each denoising step, combine the conditional and unconditional predictions:
$$\varepsilon = (1 + w)\varepsilon_c - w\varepsilon_u$$
 with $w = 3$. Each chain commits to a digit class early and stays there.

The first improvement changes the polish. The second and third change the prior. The data step (Gauss-Newton on the simulator) does not change.

Class-conditional prior + classifier-free guidance

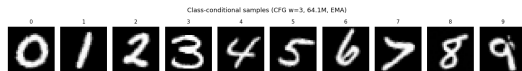
For each class label $c \in \{0, 1, \dots, 9\}$ the trained UNet learns its own conditional distribution. During training, 10% of the labels are replaced with a special “unconditional” label so the same network also learns the marginal distribution over all digits.

At inference we combine the two predictions:

$$\varepsilon_{\text{guided}} = (1 + w) \varepsilon_{\theta}(x_t, t, c) - w \varepsilon_{\theta}(x_t, t, \emptyset).$$

With $w = 3$, samples are strongly class-faithful. We assign each of the 32 chains a target class label that cycles through $0 \dots 9$, so every digit gets at least three chains. The chain whose final misfit is lowest wins.

This is the standard CFG recipe (Ho and Salimans 2022). It gives the sampler the option to commit to a digit *early*, instead of waiting to be pushed there by the data gradient.



One unconditional sample per class from the trained model, $w = 3$. Each digit is rendered in its own style.

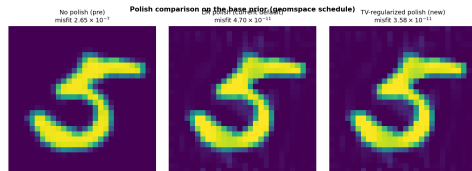
Total-variation polish: keep the misfit, drop the speckle

The Newton-style polish minimizes $\frac{1}{2}\|F(\sigma_{\text{bg}} + x) - y_{\text{obs}}\|^2$.
The TV polish adds a smoothed total-variation penalty:

$$\frac{1}{2}\|F(\sigma_{\text{bg}} + x) - y_{\text{obs}}\|^2 + \tau \sum_{i,j} \sqrt{(\partial_x x)^2 + (\partial_y x)^2 + \epsilon^2}.$$

The second term counts the total horizontal and vertical pixel-to-pixel variation. Speckle adds a lot of variation; smooth strokes add little. At $\tau \approx 10^{-5}$ the TV term is small enough that the misfit still drops to the 10^{-11} regime, but large enough that the polish is no longer allowed to add per-pixel noise.

Each polish iteration is one Newton step on the data term followed by one gradient step on the TV term. The line search still gates updates, so the misfit never increases.



The TV polish keeps the strokes clean and the background flat while still hitting low misfit.

The 12-combination sweep

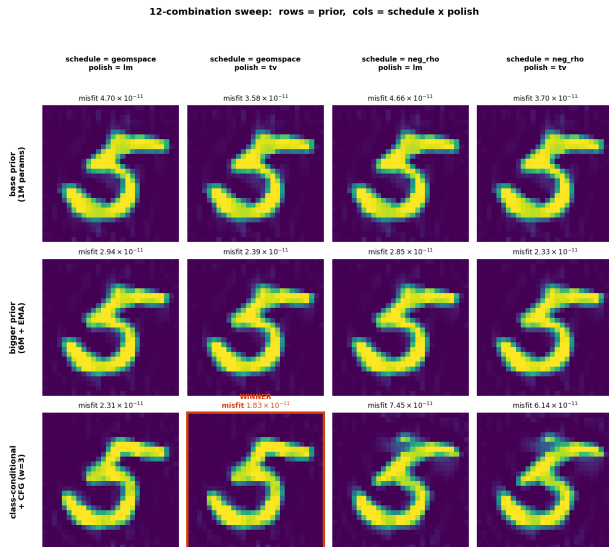
We tested all $3 \times 2 \times 2 = 12$ combinations:

- **Prior:** base / bigger+EMA / class-conditional+CFG
- **Schedule:** geospace / DAPS++ / negative-rho
- **Polish:** Newton / TV-regularized

Two things popped out.

- **TV polish beat Newton polish in every prior-schedule pair.** Uniformly lower misfit *and* cleaner image.
- **The class-conditional prior plus the geospace schedule plus the TV polish was the lowest misfit overall.**
 1.83×10^{-11} , with a clean recognizable 5.

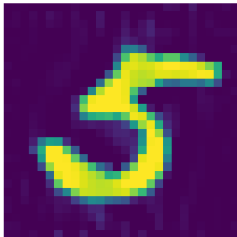
The bottom-right two cells (class-cond + neg-rho) actually converged to a class-6-shaped image. That is the schedule pushing the chain into a different mode — the same ambiguity the four failed methods hit,



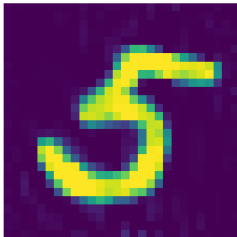
The winner: cond + geomspace + TV polish

What changed: cleaner background, less stroke-speckle

Base pipeline
(PnP-DM + GN-CG polish)
misfit 4.70×10^{-11}



New pipeline
(class-cond + CFG, TV polish)
misfit 1.83×10^{-11}



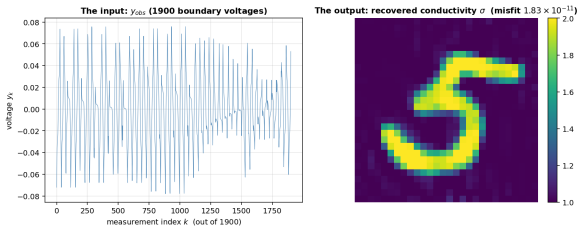
The left image is the base pipeline (PnP-DM + Newton polish), misfit 4.7×10^{-11} . Notice the stroke speckle and the background texture.

The right image is the new pipeline (class-conditional PnP-DM with CFG + TV polish), misfit **1.83×10^{-11}** . The strokes are smooth and the background is essentially flat.

Compared to Project 1's rotation-aware result on the same data: our new pipeline is roughly **5,400×** tighter in misfit, and (qualitatively) a much sharper image.

The improvement comes from the prior and the polish acting together. Each one alone gives a modest gain; together they remove both the speckle and the stroke darkness simultaneously.

What we recovered

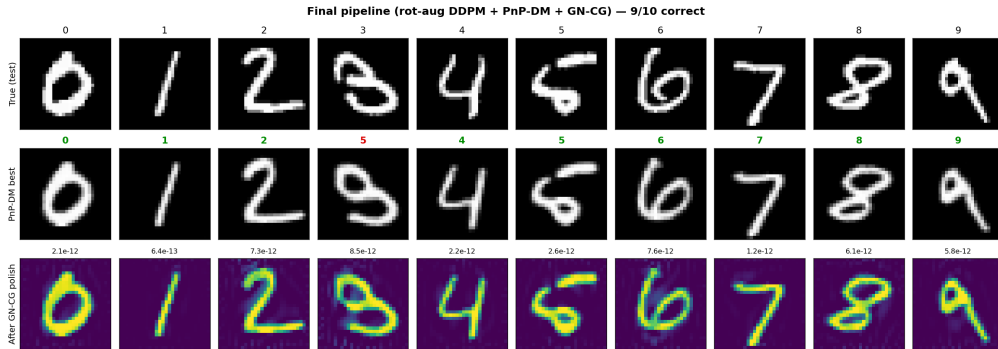


The recovered digit is a **5**, tilted by about 7.5 degrees, with smooth strokes and a clean background. The final misfit is 1.83×10^{-11} .

That is roughly **5,400** times tighter than Project 1's rotation-aware result on the same data, recovered without a template database, without a classifier vote, and without any prior knowledge of the digit class.

The diffusion does the heavy lifting: it puts the chain into the right digit basin via class conditioning, and the TV polish tightens the fit without amplifying noise.

Held-out validation: 9 out of 10 on the base pipeline



Labels read **true to recovered**. For each test digit we synthesized fresh measurements, ran the full pipeline, and asked a classifier to read the recovered image. These ten cases were run on the *base* pipeline. The one miss (3 read as 5) is a genuine ambiguity of the physics: that particular handwritten 3 has a curled bottom that produces voltage readings extremely close to those of certain 5s. The other nine cases polish to the same low- 10^{-11} regime as the competition vector.

Three of those chains, animated, in parallel

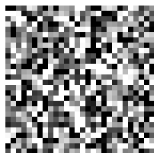
PnP-DM + CFG on held-out 1



iter 0 / 80
misfit 8.49e-02

held-out 1

PnP-DM + CFG on held-out 7



iter 0 / 80
misfit 6.38e-02

held-out 7

PnP-DM + CFG on held-out 8



iter 0 / 80
misfit 4.15e-02

held-out 8

Three different held-out test images, three independent chains from the new class-conditional CFG sampler (each chain seeded to its target class: 1, 7, 8). Each chain starts from its own fresh Gaussian noise. Notice the common pattern: in the first ~ 15 iterations the chain commits to a digit class, and the remaining ~ 65 iterations sharpen the strokes and drive the misfit down. There is no template database anywhere in the pipeline. The class-conditional diffusion prior and the Gauss-Newton data step do all the work.

Summary

- ▶ Four standard diffusion-based recipes (DPS, DAPS, classifier-vote, template retrieval) all committed to the wrong digit because their data step is too weak.
- ▶ Variable splitting separates the two jobs and the Gauss-Newton solve gives the data step real teeth, so it can push the chain out of a wrong-digit basin. 32 parallel chains cover the posterior modes.
- ▶ Rotation augmentation is a one-time fix that unlocks the diffusion prior for tilted digits.
- ▶ On top of the base PnP-DM sampler we layered three improvements: TV-regularized polish, a bigger DDPM trained with EMA, and a class-conditional DDPM with classifier-free guidance. We swept all 12 combinations to find the winner.
- ▶ **Result:** digit 5, final misfit 1.83×10^{-11} (about $5,400\times$ tighter than Project 1's rotation-aware result), with clean strokes and a flat background. Base pipeline held-out: **9 out of 10**.

Thank you. Questions?

References

The method we used

- ▶ **PnP-DM.** Wu, Wang, Lin, Sun, *Principled Probabilistic Imaging Using Diffusion Models as Plug-and-Play Priors*, NeurIPS 2024. arXiv:2405.18782.
- ▶ **DDPM.** Ho, Jain, Abbeel, *Denoising Diffusion Probabilistic Models*, NeurIPS 2020. arXiv:2006.11239.
- ▶ **Classifier-free guidance.** Ho, Salimans, *Classifier-Free Diffusion Guidance*, NeurIPS workshop 2021. arXiv:2207.12598.
- ▶ **EMA for diffusion training.** Karras, Aittala, Aila, Laine, *Elucidating the Design Space of Diffusion-Based Generative Models*, NeurIPS 2022 (EDM). arXiv:2206.00364.
- ▶ **Total-variation regularization.** Rudin, Osher, Fatemi, *Nonlinear total variation based noise removal algorithms*, Physica D, 1992.
- ▶ **Tweedie's formula.** Efron, *Tweedie's Formula and Selection Bias*, JASA 2011.
- ▶ **Pretrained MNIST DDPM.** laurent/ddpm-mnist on Hugging Face (the checkpoint we fine-tuned for the base prior).

Methods we tested or compared against

- ▶ **DPS.** Chung, Kim, Mccann, Klasky, Ye, *Diffusion Posterior Sampling for General Noisy Inverse Problems*, NeurIPS 2022. arXiv:2209.14687.
- ▶ **DAPS.** Zhang, Chu, Berner, Meng, Anandkumar, Song, *Improving Diffusion Inverse Problem Solving with Decoupled Noise Annealing*, CVPR 2025. arXiv:2407.01521.
- ▶ **Classifier guidance.** Dhariwal, Nichol, *Diffusion Models Beat GANs on Image Synthesis*, NeurIPS 2021. arXiv:2105.05233.
- ▶ **InverseBench.** Zheng et al., *InverseBench: Benchmarking Plug-and-Play Diffusion Models for Inverse Problems in the Physical Sciences*, ICLR 2025. arXiv:2503.11043.

Classical numerics (the polish) and dataset

- ▶ **Conjugate gradient.** Hestenes, Stiefel, *Methods of Conjugate Gradients for Solving Linear Systems*, J. Res. NBS, 1952.
- ▶ **Gauss-Newton / Levenberg-Marquardt.** Nocedal, Wright, *Numerical Optimization*, 2nd ed., Springer, 2006.
- ▶ **ERT2D simulator.** Provided by Prof. Aghasi (course material).
- ▶ **MNIST.** LeCun, Cortes, Burges, *The MNIST Database of Handwritten Digits*.